	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 10

Lab Title: **BST ,insertion , preorder, inorder postorder traversal**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective:understanding and im[plementation of BST and inoredr preorder postorder traversal

. BST and and real life applications

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 10

Tittle : Traversal of BST

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 4 December 24

Objective

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the
- node's key.
- The right subtree of a node contains only nodes with keys greater than the
- node's key.
- The left and right subtree each must also be a binary search tree.
- Postorder traversal Left,rigt Root
- Preorder traversal Root left rigt
- Inorder traversal Left Root Righ

❖ TASK 1

❖ **1. Write a C++ program that:**

- Creates a Binary Search Tree (BST).
- Inserts a set of integer values into the tree using the BST

❖ **insertion rules.**

❖ **Prints the tree in the following three traversal orders:**

- Inorder Traversal
- Preorder Traversal
- Postorder Traversal

====(CODE)====

```
#include<iostream>
using namespace std;
class node{
public:
int d;
node *l,*r;
node(int x){
d=x;
l=r=NULL;}
};
class BST{
public:
node* root;
int lvl;
BST(){root=NULL;lvl=0;}
node* insert(node* t,int x,int depth){
if(depth>5){
cout<<"Tree level limit (5) reached.\n";
return t;
}
if(t==NULL){
t=new node(x);
return t;
}
if(x<t->d) t->l=insert(t->l,x,depth+1);
else t->r=insert(t->r,x,depth+1);
return t;
}
```

====(CODE)====

```
29  void add(int x){
30  root=insert(root,x,1);
31  }
32  void inorder(node* t){
33  if(t==NULL) return;
34  inorder(t->l);
35  cout<<t->d<<" ";
36  inorder(t->r);
37  }
38  void preorder(node* t){
39  if(t==NULL) return;
40  cout<<t->d<<" ";
41  preorder(t->l);
42  preorder(t->r);
43  }
44  void postorder(node* t){
45  if(t==NULL) return;
46  postorder(t->l);
47  postorder(t->r);
48  cout<<t->d<<" ";
49  }
50  bool searchLast(node* t,int x){
51  if(t==NULL) return false;
52  if(t->d==x) return true;
53  if(x<t->d)
54  return searchLast(t->l,x);
55  else{
56  return searchLast(t->r,x);}
```

====(CODE)====

```
`#include<iostream>
using namespace std;
class node{
public:
int d;
node *l,*r;
node(int x){
d=x;
l=r=NULL;}
};
class BST{
public:
node* root;
int lvl;
BST(){root=NULL;lvl=0;}
node* insert(node* t,int x,int depth){
if(depth>5){
cout<<"Tree level limit (5) reached.\n";
return t;
}
if(t==NULL){
t=new node(x);
return t;
}
if(x<t->d) t->l=insert(t->l,x,depth+1);
else t->r=insert(t->r,x,depth+1);
return t;
```

====(CODE)====

```
BST b;
int ch,x,last=0;
do{
cout<<"\n===== BST MENU =====\n";
cout<<"1.Insert node\n";
cout<<"2.Search last inserted element\n";
cout<<"3.Delete node with 1 child\n";
cout<<"4.Delete leaf node\n";
cout<<"5.Delete root node \n";
cout<<"6.Display inorder\n";
cout<<"7.Display preorder\n";
cout<<"8.Display postorder\n";
cout<<"9.Exit\n";
cout<<"Enter choice: ";
cin>>ch;
if(ch==1){
cout<<"Enter value: ";
cin>>x;
last=x;
b.add(x);}
else if(ch==2){
cout<<"Searching last inserted "<<last<<"...\n";
if(b.searchLast(b.root,last)) cout<<"Found\n";
else cout<<"Not Found\n";
}
else if(ch==3){
cout<<"Enter value (1 child): ";
cin>>x;
```

====(CODE)====

```
113 b.del(x);}
114 else if(ch==4){
115 cout<<"Enter leaf node value: ";
116 cin>>x;
117 b.del(x);}
118 else if(ch==5){
119 cout<<"Deleting root: "<<(b.root? b.root->d:0)<<endl;
120 if(b.root) b.del(b.root->d);
121 }
122 else if(ch==6){
123 cout<<"Inorder: ";
124 b.inorder(b.root);
125 cout<<endl;
126 }
127 else if(ch==7){
128 cout<<"Preorder: ";
129 b.preorder(b.root);
130 cout<<endl;
131 }
132 else if(ch==8){
133 cout<<"Postorder: ";
134 b.postorder(b.root);
135 cout<<endl;
136 }
137 else if(ch!=9){
138 cout<<"Invalid\n";
139 }
140 }while(ch!=9);
141 return 0;
```

====(OUTPUT)====

MENU

C:\Users\Nextsol Pvt Ltd\Downloads\KKIK

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Display preorder
8.Display postorder
9.Exit
Enter choice:
```

Insertion first node as Root node

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Display preorder
8.Display postorder
9.Exit
Enter choice: 1
Enter value: 6
```

====(OUTPUT)====

Inorder Display

```
===== BST MENU =====
1.Insert node
2.Search last inserted elemen
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Display preorder
8.Display postorder
9.Exit
Enter choice: 6
Inorder: 2 3 5 6 7 8 9
```

Preorder

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Display preorder
8.Display postorder
9.Exit
Enter choice: 7
Preorder: 6 5 2 3 7 8 9
```

Postorder

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Display preorder
8.Display postorder
9.Exit
Enter choice: 8
Postorder: 3 2 5 9 8 7 6
```

Task2.

You are given only the preorder traversal of a Binary Search Tree (BST).
Your task is to:

- **Reconstruct the original BST from the given preorder array.**
- **After rebuilding the tree, print:**
  - **Inorder Traversal**
  - **Postorder Traversal**

====(CODE)====

```
#include<iostream>
using namespace std;

class node{
public:
int d;
node *l,*r;
node(int x){
d=x;
l=r=NULL;
}
};

node* insertBST(node* t,int x){
if(t==NULL){
t=new node(x);
return t;
}
if(x<t->d) t->l=insertBST(t->l,x);
else t->r=insertBST(t->r,x);
return t;
}

void inorder(node* t){
if(t==NULL) return;
inorder(t->l);
cout<<t->d<<" ";
inorder(t->r);
```

====(CODE)====

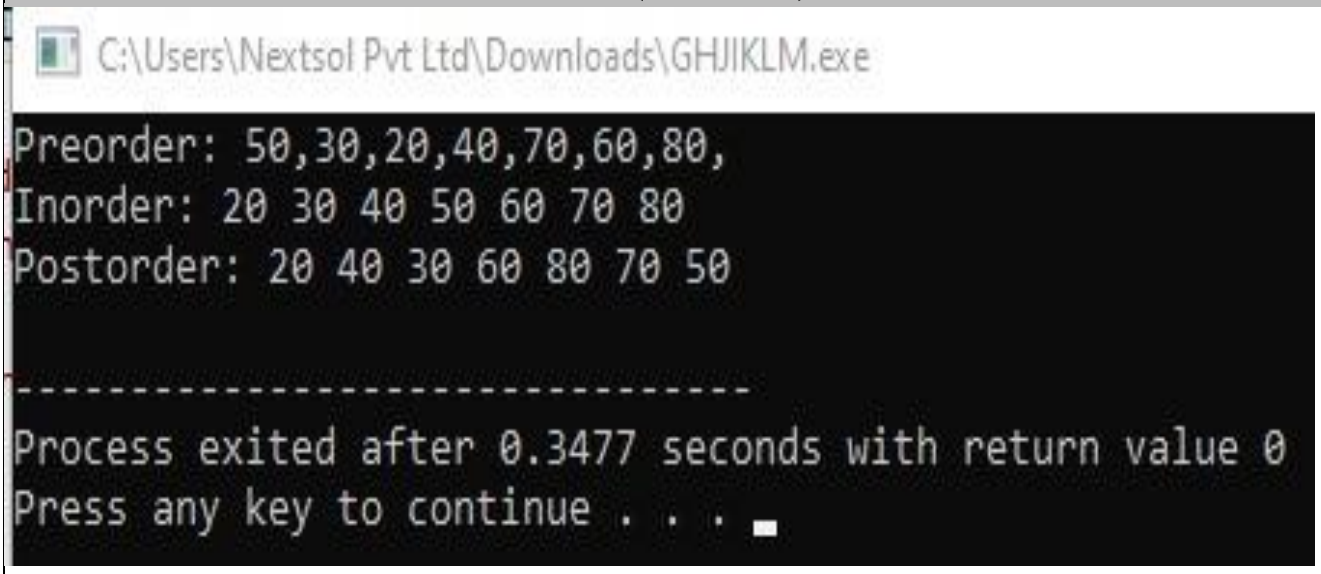
```
31  void postorder(node* t){
32  if(t==NULL) return;
33  postorder(t->l);
34  postorder(t->r);
35  cout<<t->d<<" ";
36  }
37
38  int main(){
39  node* root=NULL;
40
41  int preorder[]={50,30,20,40,70,60,80};
42  int n=7;
43
44  for(int i=0;i<n;i++){
45  root=insertBST(root,preorder[i]);
46  }
47
48  cout<<"Inorder: ";
49  inorder(root);
50  cout<<endl;
51
52  cout<<"Postorder: ";
53  postorder(root);
54  cout<<endl;
55
56  return 0;
57  }
```

====(CODE)====

```
31  void postorder(node* t){
32  if(t==NULL) return;
33  postorder(t->l);
34  postorder(t->r);
35  cout<<t->d<<" ";
36  }
37
38  int main(){
39  node* root=NULL;
40
41  int preorder[]={50,30,20,40,70,60,80};
42  int n=7;
43
44  for(int i=0;i<n;i++){
45  root=insertBST(root,preorder[i]);
46  }
47
48  cout<<"Inorder: ";
49  inorder(root);
50  cout<<endl;
51
52  cout<<"Postorder: ";
53  postorder(root);
54  cout<<endl;
55
56  return 0;
57  }
```

\

====(OUTPUT)====


C:\Users\Nextsol Pvt Ltd\Downloads\GHJIKLM.exe
Preorder: 50,30,20,40,70,60,80,
Inorder: 20 30 40 50 60 70 80
Postorder: 20 40 30 60 80 70 50

Process exited after 0.3477 seconds with return value 0
Press any key to continue . . .

Conclusion:

- In a Binary Search Tree (BST), insertion is performed by comparing the value to be inserted with the current node and placing it in the left subtree if smaller and right subtree if larger, ensuring the BST property is maintained.
- The structure of the BST depends on the order of insertion; different insertion sequences can produce different tree shapes even with the same values.
- Inorder traversal of a BST visits nodes in ascending sorted order, making it useful for displaying sorted data.
- Preorder traversal visits the root first, then the left subtree, and then the right subtree, which is useful for copying the tree or creating a prefix expression.
- Postorder traversal visits the left subtree, then the right subtree, and finally the root, making it useful for deleting the tree or evaluating postfix expressions.
- Traversal methods provide different ways to process or display tree data, and each has specific applications in algorithms and data processing.
- Understanding BST insertion and traversal is fundamental for efficient searching, sorting, and organizing hierarchical data in computer scienc